TRABAJO PRÁCTICO – 1

Base de Datos Relacionales y No Relacionales

Alumno: Ignacio Figueroa – 45.406.120

Tecnicatura Universitaria en Programación – UTN

Materia: Base de Datos 2

Comisión: 7

Objetivo:

Comprender y aplicar de manera práctica las sentencias fundamentales de SQL y las estructuras de modelado documental, para finalmente identificar, analizar y la elección del modelo de base de datos más apropiado (Relacional SQL o No Relacional NoSQL) basándose en los requisitos del sistema.

Aclaracion:

Para la creación de tablas en SQL utilicé Neon Postgres para simplicidad, ya que desinstalé MySQL de mi equipo local.

1) Cree las siguientes tres tablas asegurando la integridad referencial.

PRODUCTOS (id_producto, nombre, precio, stock)

CLIENTES (id_cliente, nombre, email, fecha_registro)

PEDIDOS (id_pedido, cliente_id, fecha, total)

```sql
CREATE TABLE CLIENTES (
    id_cliente INT PRIMARY KEY,
    nombre VARCHAR(100),
    email VARCHAR(100),
    fecha_registro DATE
);

CREATE TABLE PRODUCTOS (
    id_producto INT PRIMARY KEY,
    nombre VARCHAR(100),
    precio DECIMAL(10,2),
    stock INT
);

CREATE TABLE PEDIDOS (
    id_pedido INT PRIMARY KEY,
    cliente_id INT,
    producto_id INT,
    fecha DATE,
    total DECIMAL(10,2),
    FOREIGN KEY (cliente_id) REFERENCES CLIENTES(id_cliente),
    FOREIGN KEY (producto_id) REFERENCES PRODUCTOS(id_producto)
);
```

2) Inserts

```sql
INSERT INTO CLIENTES VALUES (1, 'Juan Perez', 'juan@mail.com', '2025-01-10');

INSERT INTO PRODUCTOS VALUES
(1, 'Mouse', 150, 10),
(2, 'Teclado', 200, 15);
```

3) Update

```sql
UPDATE PRODUCTOS
SET stock = 5
WHERE id_producto = 1;
```

5) Consultas

d) **Productos > 100**

```sql
SELECT * FROM PRODUCTOS
WHERE precio > 100;
```

e) Nombre del cliente + fecha del pedido

```sql
SELECT C.nombre, P.fecha
FROM PEDIDOS P
JOIN CLIENTES C ON P.cliente_id = C.id_cliente;
```

f) Cantidad de pedidos por cliente

```sql
SELECT cliente_id, COUNT(*) AS cantidad_pedidos
FROM PEDIDOS
GROUP BY cliente_id;
```

PARTE 2 - NoSQL (MongoDB / JSON)

a) Ejemplos de documentos (UNA sola colección)

```
// Perfil Regular
{
  "_id": 1,
  "nombre": "Ignacio",
  "email": "nacho@mail.com",
  "intereses": ["programacion", "viajes"],
  "direccion": {
    "ciudad": "Buenos Aires",
    "domicilio": "Libertad 1500"
  }
}

// Perfil Premium
{
  "_id": 2,
  "nombre": "Maria",
  "email": "maria@mail.com",
  "fecha_suscripcion": "2025-02-01",
  "limite_credito": 5000
}

// Perfil Corporativo
{
  "_id": 3,
  "razon_social": "TechCorp",
  "email": "contacto@techcorp.com",
  "empleados": 50
}
```

b) Polimorfismo (explicación corta para aprobar)

El polimorfismo en NoSQL permite que distintos tipos de documentos con estructuras diferentes convivan dentro de una misma colección. En este caso, los perfiles Regular, Premium y Corporativo tienen distintos campos, pero todos se almacenan en la colección "Perfiles", adaptándose a cada tipo de usuario sin necesidad de un esquema fijo.

c) Anidamiento (explicación clara)

El anidamiento de datos en NoSQL consiste en incluir estructuras dentro de otras, como objetos o arrays dentro de un documento. En el perfil Regular, los campos "intereses" (array) y "direccion" (subdocumento) son ejemplos de anidamiento, ya que permiten organizar información relacionada dentro de un mismo documento, evitando múltiples tablas o relaciones como en SQL.

Conclusion

La elección entre SQL y NoSQL depende de los requisitos del sistema. SQL es ideal para datos estructurados y relaciones claras, mientras que NoSQL ofrece flexibilidad para manejar datos heterogéneos y escalabilidad.